Lia
The mascot. **Scale** is the product name; Lia is not a synonym for the product and does not appear in code or generated artifacts. Historical prose used Lia as the prototype name. Definition: README.typ. Used in: README.typ.

Set
A collection of values. In Scale the values are tensors, so a set is a collection of tensor values; membership is structural. An anonymous set is written `[ … ]`; a named set is a value set. The store holds the members as store entries. Definition: components/prelude/evaluator/value-sets.fs. Used in: set-algebra.fs, dispatch.fs, development/scale/tests.fs.

ValueSet
A named set: a seal (a text name), an extension (member identities), and an optional narrowing quotation or runtime guard-style constructor. The code type is `EvaluatorValueSets.ValueSet`. A value set is not an ordering; the prose term is *sort*. Definition: components/prelude/evaluator/value-sets.fs. Used in: dispatch.fs, set-algebra.fs, type-lattice.fs.

Cell system
The layer above the metaobject substrate: classes, slots, instances, class precedence, nominal dispatch, method combination, closures, and metaclasses. The encoding landed as D44, closures/lexical scope as D45, classes/slots/instances as D46, precedence/nominal dispatch as D47, method qualifiers and combination as D48, and metaclasses as D49; conditions and restarts (D50) landed. The surface stays Scale Basic English. Definition: docs/cell-system.typ (D43), cell-system.fs (D44 encoding), class-system.fs (D46). Used in: development/scale/tests.fs A–G landed.

Cell
A cell is an instance: stable address identity plus appended slot state. A cell is not a tensor. A tensor is the structural numeric value, compared by data and shape; a cell is a nominal instance, compared by identity, whose slot values are tensors. A class is the cell's class, not its data type; two cells with equal slot values are still distinct cells. Definition: cell-system.fs (D44 encoding), class-system.fs (D46). Used in: development/scale/tests.fs. The layer above the metaobject substrate: classes, slots, instances, class precedence, nominal dispatch, method combination, closures, and metaclasses. The encoding landed as D44, closures/lexical scope as D45, classes/slots/instances as D46, and precedence/nominal dispatch as D47; the conditions and restarts (D50) landed. The surface stays Scale Basic English. Definition: docs/cell-system.typ (D43), cell-system.fs (D44 encoding), class-system.fs (D46). Used in: development/scale/tests.fs landed.

Cell system, Scale versus the typical
The typical class-based system is a type declaration: a class fixes its slots and methods at compile time, an instance is a mutable cell, and dispatch is usually on one class. Scale's cell system is a return, in spirit, to Alan Kay's original framing: cells communicate, binding is late, and the definition is data. Classes remain, but a class is an image-resident metaobject, a slot is a definition with a value set, an instance is stable identity plus appended state, and dispatch is nominal over classes and precedence (D43–D50). This is a Lisp-flavoured cell system: definitions are store entries, a new class, slot, method, combination, or metaclass is data, the REPL is the development loop, and redefinition appends instead of rewriting. It borrows CLOS's generic functions, method combination, and metaobject protocol (D16, D17) without a reflective tower. Live coding and flexibility are the point: redefinition is an append, history can roll back, and a domain that does not opt in is unaffected. The remaining behavior is deferred; the encoding landed as D44, closures as D45, classes/slots/instances

as D46, and precedence/nominal dispatch as D47. The verification boundary is unchanged: the cell system is a runtime feature, not an SMT theory.

Aspect	Typical class-based	Scale cell system
Class	compile-time nominal type	image-resident definition; nominal identity plus precedence
Slot	fixed layout	definition as data; a value set; appended state per write
Instance	mutable cell	stable identity plus appended state
Dispatch	single dispatch on the class	multiple dispatch over classes and value sets
Methods	attached to a class	generic functions; qualifiers and combination are data
Redefinition	recompile and restart	append a new definition; history and rollback
Development	compile and run	live REPL over the image

Definition: docs/cell-system.typ (D43–D50). Used in: README.typ A–G landed.

Cell system versus Smalltalk, CLOS, and Ruby/Python

Scale shares their late-bound, image-based spirit, but its class is image data, dispatch is a generic-function protocol over classes and value sets, instances are stable identity plus appended state, and the image is append-only.

1. **Smalltalk** is the closest in spirit: classes are cells, messages are late-bound, and the image is live. Scale differs in its append-only image (no host cell graph), in multiple dispatch through generic functions rather than method lookup on the receiver, and in the verification boundary. There is no `self`/`super` or `become:`; redefinition appends.
2. **CLOS** is the closest in structure: generic functions, multiple dispatch, method combination, and the MOP are the model. Scale borrows those but keeps a fixed meta-level (no reflective tower), uses an append-only image instead of mutable instances, and has no Lisp macros or condition system yet.
3. **Ruby** has open classes and modules; Scale's classes are image entries, so a redefinition appends rather than reopens. Ruby methods live on the receiver's class/module chain; Scale dispatches through generic functions over all arguments and value sets. There is no `method_missing` silent fallback.
4. **Python** uses C3 MRO, bound methods, `self`, descriptors, and a mutable `__dict__`. Scale has a nominal class, a C3 deterministic precedence, and declared slots only; a missing slot or an undecidable dispatch is named. Both support multiple inheritance, but Scale's dispatch is multiple and nominal rather than single-dispatch on the first argument.

The load-bearing difference is the image: their cells live in a mutable host heap, Scale's live in an append-only tensor image. That is why redefinition, instance state, and history work differently.
Definition: docs/cell-system.typ (D43–D50). Used in: README.typ A–G landed.

Class

A class is an image-resident metaobject that owns a name, a superclass list, a slot list, a metaclass, and a method-combination policy. Its extension is a value set the cell-system class is not the old `ClassTag` verdict (now ValueTag). The `class` word declares one in the lab surface; the Basic English

declaration grammar is still deferred. Definition: cell-system.fs (D44 encoding), class-system.fs (D46). Used in: development/scale/tests.fs.

Slot
A slot is an image-resident definition with a name, a value set, an optional default, and an initialization rule. The `slot` word declares one; a slot's value is checked against its set. Definition: cell-system.fs (D44 encoding), class-system.fs (D46). Used in: development/scale/tests.fs.

Instance
An instance is a value with stable address identity and an appended state: the identity entry is written once, and every slot write appends a new state entry, exactly like channel state (D27). `create-instance` creates one; `slot-value` reads the latest state and `set-slot-value` appends a new one. Definition: cell-system.fs (D44 encoding), class-system.fs (D46). Used in: development/scale/tests.fs.

Class precedence
Class precedence is a deterministic linearization of the superclass graph. Scale uses C3; a cycle is `ClassCycle` and a conflicting order is `InconsistentPrecedence`. Definition: class-system.fs (D47). Used in: dispatch.fs, development/scale/tests.fs.

Class dispatch
Dispatch on a class is nominal (class identity plus precedence), not extensional, so it is stable as instances are added. Class specializers use precedence inclusion; a class/value-set tie is ambiguous. Definition: dispatch.fs (D47), class-system.fs (D47). Used in: development/scale/tests.fs.

Method combination
Methods gain a qualifier, and a method combination orders them. The qualifiers are Primary, Before, After, and Around. Definition: cell-system.fs (D44 encoding). Used in: development/scale/tests.fs landed.

Closure
Closures are image-resident code cells with an image-resident environment chain, so lexical scope does not become host state. Lexical scope landed as D45: `let` creates a frame, `lambda` captures it, and `apply` runs in it and restores the caller. Definition: cell-system.fs (D44 encoding), closure.fs (D45). Used in: development/scale/tests.fs.

Environment frame
An environment frame is an image-resident lexical frame with a parent and name/value pairs. The current frame is execution state, never serialized (D45). Definition: cell-system.fs (D44 encoding), closure.fs (D45). Used in: development/scale/tests.fs.

Metaclass
Metaclasses make classes instances of a class, with a finite class-of closure (the cell query is `cell-class`) terminating at a root metaclass. The old classification word `class-of` is now `value-of`. Definition: cell-system.fs (D44 encoding). Used in: development/scale/tests.fs landed.

Set-operation protocol
The six set words are image-resident generic functions: `member`, `subset`, `union`, `intersect`, `except`, and `complement`. Each has a default method specialized on the empty `set` declaration; a set entry is classified as `set` by the dispatcher, and the method body calls the kernel primitive of the same operation (`set-member`, `set-subset`, and so on). Replacing a method is an image change, not an F# change. Definition: dispatch.fs, value-sets.fs. Used in: evaluator.fs, development/scale/tests.fs.

Tensor
The only semantic value: a rank-n array of numbers with a shape. A scalar is a rank-0 tensor. Tensors are compared structurally; two tensors with the same data and shape are the same set member.

A tensor is not a cell. A tensor has no address identity and no appended state; a cell has both, and its slot values are tensors. Structural tensor equality is not cell identity: two cells with equal slot tensors remain distinct cells. Definition: tensor.fs. Used in: store.fs, value-sets.fs, smt.fs.

EntryIdentity
A nominal address of an entry in the store. It is a rank-0 tensor projection, not a host pointer; the store is the only authority. Definition: types.fs. Used in: store.fs, dispatch.fs, persistence.fs.

Store
The append-only image of every tensor and definition. It is not the evaluation stack: the store holds data, the stack holds control state. The old name was `Image`. Definition: store.fs. Used in: persistence.fs, evaluator.fs, development/scale/tests.fs.

Stack
The evaluator's control stack: push, pop, and peek of EntryIdentitys. It is not the store. Definition: stack.fs. Used in: evaluator.fs, components/prelude/evaluator/stack.fs.

Lexicon
The store's tensor table: address to tensor. The vocabulary is stored separately, as image entries. Definition: lexicon.fs. Used in: store.fs, types.fs.

Vocabulary
The store-resident word list. A VocabularyEntry is a word's token, form, category, active flag, and implementation reference; the runtime word metaobject projects it. Definition: vocabulary.fs, vocabulary-in-image.fs. Used in: tokenizer.fs, parser.fs, evaluator.fs.

VocabularyEntry
A word's image projection: token, form, word category, active flag, and implementation name. The runtime word metaobject projects it. Definition: types.fs, vocabulary-in-image.fs. Used in: vocabulary.fs, tokenizer.fs, parser.fs.

Word
A word metaobject: `[WordTag; token; form; category; active; implementation]`. It makes a new word family data rather than an F# case. Definition: components/prelude/evaluator/word.fs. Used in: evaluator.fs, vocabulary-in-image.fs, development/scale/tests.fs.

WordCategory
The role of a vocabulary word: TypeName, FieldName, FunctionName, Copula, Comparison, or one of the other closed-class roles. It is not the value tag of a value. Definition: types.fs. Used in: vocabulary.fs, parser.fs, components/prelude/evaluator/word.fs.

ValueTag
The image tag `-12.0` of a value classification verdict: `[ValueTag; status; payload…]`. Statuses are Unknown, a definite value set, Ambiguous, Undecided, and Metaobject. The value verdict is computed by valueVerdictOf and returned by the value-of surface word. Definition: types.fs, components/prelude/evaluator/dispatch.fs. Used in: dispatch.fs, development/scale/tests.fs.

ValueVerdict
The internal classification answer: `ValueOf of #g(<g-value-set>)[ValueSet]`, `ValueUnknown`, `ValueAmbiguous of #g(<g-value-set>)[ValueSet] list`, or `ValueUndecided of string`. It is not a cell-system class. Definition: components/prelude/evaluator/dispatch.fs. Used in: dispatch.fs, development/scale/tests.fs.

Membership
The answer to "is this value in this value set?": `Member`, `NotMember`, or `Undecided reason`. An undecidable predicate is never a silent false. Definition: components/prelude/evaluator/value-sets.fs. Used in: dispatch.fs, set-algebra.fs.

EntryKind
The kind of a tagged store entry: word, generic function, method, channel, channel state, channel event, history step, predicate value set, runtime guard, or constructor. It is not a value classification. Definition: components/prelude/evaluator/dispatch.fs. Used in: dispatch.fs, development/scale/tests.fs.

EvaluationState
The execution state: the stack, the `Term list` program and program counter, roots into the store, scheduler state, and ownership state. It is never serialized. The program representation is Term IR. Definition: types.fs. Used in: evaluator.fs, scheduler.fs, persistence.fs.

Quotation
A code value in braces, not token text: `[QuoteTag; documentId]` naming the canonical term-program document for the body. It runs on a fresh stack with a reduction budget. Control words, predicate sets, runtime guards, wrappers, and method bodies use it. See code value and Term IR. Definition: control.fs, code.fs. Used in: value-sets.fs, runtime-guard.fs, dispatch.fs.

Scheduler
The cooperative actor scheduler and its per-process state: spawn, yield, halt, resume, kill, channels, and event waiters. Definition: components/prelude/evaluator/scheduler.fs. Used in: evaluator.fs, development/scale/tests.fs.

Actor
A scheduled process with an identity, parent, children, status, and its own EvaluationState. Definition: types.fs. Used in: scheduler.fs, ownership.fs.

OwnershipState
Per-run linear ownership: owners and consumed addresses. It is execution state, never serialized. Definition: components/prelude/evaluator/ownership.fs. Used in: evaluator.fs, development/scale/tests.fs.

Evaluator
The stack-machine entry point: `executeWord` dispatches a token through the built-ins, word metaobjects, generic functions, user words, and the vocabulary. Definition: components/prelude/evaluator/evaluator.fs. Used in: development/scale/tests.fs, bases/scale-cli/entry.fs, properties.fs.

EvaluatorValueSets
The evaluator slice that owns value sets, structural set operations, membership, and predicate/constructor narrowing. Definition: components/prelude/evaluator/value-sets.fs. Used in: dispatch.fs, set-algebra.fs, development/scale/tests.fs.

EvaluatorDispatch
The multiple-dispatch slice: generic functions, methods, type-first selection, and the value tag / entry kind queries. Definition: components/prelude/evaluator/dispatch.fs. Used in: evaluator.fs, development/scale/tests.fs.

EvaluatorSetAlgebra
The slice for the `set`, `add-member`, `and`, `or`, `nor`, and `is` words over image-resident value sets. The old name was `Vivid`. Definition: components/prelude/evaluator/set-algebra.fs. Used in: evaluator.fs, development/scale/tests.fs.

EvaluatorChannelEvent
The ChannelEvents event slice: `send-event`, `receive-event`, `choose`, `wrap`, `guard`, `with-fallback`, and `sync` over store-resident ChannelEvent values. Definition: components/prelude/evaluator/channel-event.fs. Used in: evaluator.fs, development/scale/tests.fs.

EvaluatorChannel
The channel slice: `channel`, `send`, `receive`, and `close`. A blocked receive is a waiter, not an error. Definition: components/prelude/evaluator/channel.fs. Used in: evaluator.fs, development/scale/tests.fs.

ChannelEvent
A ChannelEvents communication event: send, receive, choose, wrap, guard, or with-fallback. `sync` commits one; a blocked sync is a waiter, not an error. Definition: components/prelude/evaluator/channel-event.fs. Used in: evaluator.fs, channel-event.fs, development/scale/tests.fs.

EvaluatorRuntimeGuard
The runtime guard metaobject slice: `runtime-guard-define` and `runtime-guard-holds`; the guard's quotation runs through the shared quotation run. Definition: components/prelude/evaluator/runtime-guard.fs. Used in: evaluator.fs, development/scale/tests.fs.

QuotationRun
The shared probe that runs a quotation on one input with a reduction budget and no actor words; membership and runtime guards both use it. The old name was `EvaluatorProbe`. Definition: components/prelude/evaluator/quotation-run.fs. Used in: value-sets.fs, runtime-guard.fs.

HistoryStep
One executed term in the append-only event stream: program counter metadata, the current term, the stack before it ran, and the program it resumes. The term and program are code values, not token text. `history` and `rollback` read and restore steps. Definition: components/prelude/evaluator/history.fs. Used in: evaluator.fs, development/scale/tests.fs.

UnificationTerm
A first-order term: a variable (`?name`), an atom, a compound (`-2` tagged), or a typed term (`-3` tagged). `unify`, `resolve`, `term`, and `typed` operate on them. The old name was `Term`; current `Term` means a Term IR term and is not this type. Definition: components/prelude/evaluator/environment.fs. Used in: dispatch.fs, development/scale/tests.fs.

ProvenClosure
The proved guard closure for a refinement: `pre ∧ guard ∧ ¬post` is unsatisfiable. The old name was `Closure`. Definition: refiner.fs. Used in: refiner.fs, repair.fs, development/scale/tests.fs.

ScalarRead
The projection helpers that read a rank-0 tensor as a number or a text tensor as a string. The old name was `Lace`. Definition: scalar-read.fs. Used in: evaluator.fs, bases/scale-cli/entry.fs.

VocabularyList
The read-only vocabulary browser: `showVocabulary` and `explainWord`. The old name was `Bag`. Definition: vocabulary-list.fs. Used in: bases/scale-cli/entry.fs, demo.fs.

Smt
The in-process Z3 bridge: Satisfiability Modulo Theories (SMT) encoding, satisfaction, and discharge of obligations. It returns `Proved`, `Falsified`, `Unknown`, or `Unavailable`. Definition: smt.fs. Used in: refiner.fs, repair.fs, decision-table.fs.

SolverAssignment
A solver model read back as domain values, so tests, bundles, and decision tables do not hold a Z3 context. The old type name was `Model`. Definition: smt.fs. Used in: refiner.fs, repair.fs, decision-table.fs.

Refinement
A `.scale` refinement: a refinement guard, its origin, and its counterexample. It is append-only in the document. Definition: scale-file.fs. Used in: refiner.fs, emitter.fs, repair.fs.

RefinementGuard
A typed single-field guard from a `.scale` refinement: `GreaterThan`, `AtLeast`, `LessThan`, `AtMost`, `IsTrue`, or `IsFalse`. It is the verification projection, separate from the runtime guard metaobject. Definition: scale-file.fs. Used in: refiner.fs, type-lattice.fs, emitter.fs.

FieldType
A verification field's domain type: `BooleanType`, `RealType`, `IntegerType`, or `NaturalType`. It is not a value set it is the lattice's field coordinate.

- `BooleanType` is a truth value. Use it for flags, predicates, and any field whose guards are only `is true` or `is false`. It has no numeric threshold.
- `IntegerType` is an exact, discrete number on the SMT `Int` sort. Use it for counts, indices, versions, retries, and token budgets: quantities where a fractional value is meaningless.
- `NaturalType` is an `IntegerType` constrained to be at least zero. Use it when the same discrete quantity cannot be negative.
- `RealType` is a continuous number on the SMT `Real` sort. Use it for measurements and ratios where fractional bounds matter: positions, speeds, temperatures, probabilities, and normalized scores.

The emitted F# type is `bool` for Boolean, `int` for Integer and Natural, and `float` for Real. A mixed Real and integer domain selects `QF_LIRA`; Real alone selects `QF_LRA`; the integer-only path stays `QF_LIA`.

Definition: domain-expression.fs, type-lattice.fs. Used in: smt.fs, emitter.fs, refiner.fs.

TypeLattice
The one verification coordinate system shared by Z3, Hedgehog, and the refiner: fields, finite coordinates, and narrowing guards. Definition: type-lattice.fs. Used in: smt.fs, refiner.fs, emitter.fs.

Type inference
Scale does not have a general type-inference engine hidden somewhere. What it has is **signature-directed elaboration**: declarations and word contracts supply the types, the checker propagates

and verifies them, and an expression with no usable signature is refused rather than guessed. That distinction matters because "inference" sounds like the compiler can recover any type from any program, which is not true here.

There are three sources of type information, and only one of them is inference:

1. **Scale word signatures.** The signature and effect table is the authority. A call checks its arguments and result against the called word; a local binding is checked against the words that consume it. This is local constraint propagation over known signatures, not general Hindley–Milner inference. If the constraints do not determine a type, the result is a named diagnostic, not a default.
2. **The F# compiler at the package boundary.** For F# definitions and the F# Prelude port, the F# compiler has already done the inference. Scale reads the result through **FSharp.Compiler.Service** (FCS): the checked symbol, `FullType`, curried parameter groups, return parameter, generic parameters, and constraints. This is compiler read-off. It is not Scale inferring an F# binding, and it does not transfer to arbitrary Scale syntax.
3. **Runtime value tags.** Tensor tags validate a value at execution time. They are a safety check, not a type-inference source; Scale does not observe a running program and then infer its static type.

The phrase "inference without a dedicated implementation" is therefore true only in a narrow sense. F# inference is free because the F# compiler already implements it and FCS exposes the answer. Scale inference is not free. P4 now supplies the missing implementation: P4a is a host-side first-order type constraint and unification core with schemes, substitutions, an occurs-check, exact effect sets, generalization, instantiation, and named diagnostics. P4b connects it to `ExpressionAst` and the word-signature table; P4c integrates the result into lowering and source-ranged diagnostics. A faithful F# projection checked by FCS is an alternative only if it preserves effects, stack behavior, source ranges, and Scale-only words; it is not a substitute by default.

The durable rules:

1. Prefer an explicit signature at a boundary; inference is a convenience inside a checked body.
2. The signature table, not the runtime, is authoritative.
3. An unknown or ambiguous type is a diagnostic; never silently choose one.
4. Do not infer a tuple or a function from the observed stack shape.
5. Inferred facts are elaboration-time hints, never portable semantics.
6. Higher-order type unification is not needed for ordinary higher-order programming; function types and schemes cover that. It is reserved for a future type-level lambda or higher-kinded feature.

Definition: docs/type-inference.typ, type-inference.fs, docs/decision-records.typ (D173), docs/function-values.typ, docs/language-feature-ir.typ. Used in: docs/function-values.typ, docs/prelude-port.typ, FieldType, TypeLattice, Expression IR.

Refiner
The candidate-guard synthesizer: it consumes a falsification and a proven closure oracle and returns a refinement. Definition: refiner.fs. Used in: demo.fs, repair.fs, development/scale/tests.fs.

Repair
The fixed-spec acceptance gate: it turns a witness into a versioned repair bundle and accepts a patch only when the target proves and no passing witness regresses. Definition: repair.fs. Used in: bases/scale-cli/entry.fs, cli.fs, development/scale/tests.fs.

DecisionTable
The optional symbolic table of guarded rows; `check` proves completeness and, under `Unique`, disjointness. It is a specification refinement, not the emitter. Definition: decision-table.fs. Used in: docs/decision-models.typ, development/scale/tests.fs.

Parser
The Prose sentence parser (historically Basic English): quantifier, subject, optional relative clause, verb, full stop. It refuses unsupported sentences by name. The prose parser is a separate surface parser for match, calls, pipelines, and partial application. Definition: parser.fs. Used in: emitter.fs, development/scale/tests.fs, properties.fs.

Persistence
The binary image format: it serializes the store data only; roots and ownership are execution state and are re-threaded by the reader. Definition: persistence.fs. Used in: bases/scale-cli/entry.fs, development/scale/tests.fs.

Surface vocabulary
The words a user types. Each row links to its definition in the dispatcher and the slice that owns it. *These are the current spellings.*

Word	Purpose	Definition	Used in
`set`	Declares a named value set with an empty extension.	set-algebra.fs	development/scale/tests.fs
`add-member`	Adds a value to a value set refuses a predicate or constructor set.	set-algebra.fs	development/scale/tests.fs
`seal`	Names an anonymous set with a text seal.	value-sets.fs	development/scale/tests.fs
`predicate-seal`	Names a set narrowed by a predicate quotation.	value-sets.fs	development/scale/tests.fs
`constructor`	Declares a reusable named constructor quotation.	value-sets.fs	development/scale/tests.fs
`constructor-seal`	Builds a named value set from a constructor and a base extension.	value-sets.fs	development/scale/tests.fs
`unname`	Drops a seal, keeping the extension.	value-sets.fs	development/scale/tests.fs
`subset`	Set-operation method: structural subset test over a value set.	dispatch.fs + value-sets.fs	development/scale/tests.fs
`union`	Set-operation method: structural union of two finite extensions.	dispatch.fs + value-sets.fs	development/scale/tests.fs

Word	Purpose	Definition	Used in
`intersect`	Set-operation method: structural intersection of two finite extensions.	dispatch.fs + value-sets.fs	development/ scale/tests.fs
`except`	Set-operation method: structural difference of two finite extensions.	dispatch.fs + value-sets.fs	development/ scale/tests.fs
`member`	Set-operation method: asks whether a value is in a value set returns a membership verdict.	dispatch.fs + value-sets.fs	development/ scale/tests.fs
`same-shape`	Structural equality of two extensions.	value-sets.fs	development/ scale/tests.fs
`same-name`	Nominal equality of two seals.	value-sets.fs	development/ scale/tests.fs
`sort-of`	Asks whether a value carries a seal.	value-sets.fs	development/ scale/tests.fs
`merge`	Seal-preserving union; refuses a seal mismatch.	value-sets.fs	development/ scale/tests.fs
`complement`	Set-operation method: complements over a declared domain; refuses an anonymous set.	dispatch.fs + value-sets.fs	development/ scale/tests.fs
`is`	Membership test used by the sentence surface.	evaluator.fs	development/ scale/tests.fs
`filter`	Keeps the members a predicate accepts.	value-sets.fs	development/ scale/tests.fs
`value-of`	Pushes the value's value tag / value verdict.	dispatch.fs	development/ scale/tests.fs
`word`	Declares an image-resident word metaobject.	word.fs	development/ scale/tests.fs
`generic`	Declares an image-resident generic function.	dispatch.fs	development/ scale/tests.fs
`method`	Adds a method to a generic function.	dispatch.fs	development/ scale/tests.fs
`term`	Builds a compound unification term.	environment.fs	development/ scale/tests.fs
`typed`	Wraps a term with a nominal seal.	environment.fs	development/ scale/tests.fs
`bind`	Binds a name globally in the store-resident binding table; `let` is the frame-local form.	environment.fs	development/ scale/tests.fs

Word	Purpose	Definition	Used in
`lookup`	Resolves a name through the binding table.	environment.fs	development/scale/tests.fs
`unify`	Structurally unifies two unification terms with an occurs-check.	environment.fs	development/scale/tests.fs
`resolve`	Zonks a unification term through the substitution chain.	environment.fs	development/scale/tests.fs
`runtime-guard-define`	Declares an image-resident runtime guard.	runtime-guard.fs	development/scale/tests.fs
`runtime-guard-holds`	Runs a guard quotation against a value; returns 1 or 0.	runtime-guard.fs	development/scale/tests.fs
`channel`	Creates a store-resident channel.	channel.fs	development/scale/tests.fs
`send`	Sends a message to a channel.	channel.fs	development/scale/tests.fs
`receive`	Receives a message; a blocked receive is a waiter, not an error.	channel.fs	development/scale/tests.fs
`close`	Closes a channel and wakes its waiters.	channel.fs	development/scale/tests.fs
`send-event`	Builds a send channel event.	channel-event.fs	development/scale/tests.fs
`receive-event`	Builds a receive channel event.	channel-event.fs	development/scale/tests.fs
`choose`	Selects one ready event from a list.	channel-event.fs	development/scale/tests.fs
`wrap`	Runs a quotation over the selected event value.	channel-event.fs	development/scale/tests.fs
`guard`	Suspends until the guard quotation holds.	channel-event.fs	development/scale/tests.fs
`with-fallback`	Commits a fallback event when the main event loses.	channel-event.fs	development/scale/tests.fs
`sync`	Commits one event; a blocked sync is a waiter.	channel-event.fs	development/scale/tests.fs
`spawn`	Starts a child actor in the scheduler.	scheduler.fs	development/scale/tests.fs
`yield`	Yields control to another actor.	scheduler.fs	development/scale/tests.fs
`halt`	Halts the current actor.	scheduler.fs	development/scale/tests.fs

Word	Purpose	Definition	Used in
`actors`	Lists the actors and their states.	scheduler.fs	development/scale/tests.fs
`resume`	Resumes a halted actor.	scheduler.fs	development/scale/tests.fs
`kill`	Terminates an actor killing self is a named refusal.	scheduler.fs	development/scale/tests.fs
`own`	Takes linear ownership of a value for the run.	ownership.fs	development/scale/tests.fs
`borrow`	Checks a read against ownership.	ownership.fs	development/scale/tests.fs
`consume`	Ends a value's life for the run.	ownership.fs	development/scale/tests.fs
`release`	Gives ownership up.	ownership.fs	development/scale/tests.fs
`history`	Reports the history steps.	history.fs	development/scale/tests.fs
`rollback`	Restores a history step.	history.fs	development/scale/tests.fs
`bookmark`	Records a named history step.	bookmark.fs	development/scale/tests.fs
`jump`	Restores through a bookmark.	bookmark.fs	development/scale/tests.fs
`copyBookmark`	Duplicates a bookmark value.	bookmark.fs	development/scale/tests.fs
`showBookmark`	Renders a bookmark's saved location.	bookmark.fs	development/scale/tests.fs
`if`	Runs one of two quotations.	control.fs	development/scale/tests.fs
`apply`	Runs a quotation with the current frame, or a closure with its captured frame.	control.fs	development/scale/tests.fs
`let`	Binds a name in a new child frame and makes it current.	closure.fs	development/scale/tests.fs
`lambda`	Captures the current frame in a closure.	closure.fs	development/scale/tests.fs
`class`	Declares a class the name lists are the lab surface.	class-system.fs	development/scale/tests.fs
`slot`	Declares a slot with a named value set.	class-system.fs	development/scale/tests.fs

Word	Purpose	Definition	Used in
`create-instance`	Creates a instance with stable identity and an initial state.	class-system.fs	development/scale/tests.fs
`slot-value`	Reads an instance's latest slot value; unbound and missing are named refusals.	class-system.fs	development/scale/tests.fs
`set-slot-value`	Validates a value and appends a new instance state.	class-system.fs	development/scale/tests.fs
`class-precedence`	Pushes the class names in C3 precedence order.	class-system.fs	development/scale/tests.fs
`subclass-of`	Answers whether one class is the other or a subclass.	class-system.fs	development/scale/tests.fs
`instance-of`	Answers whether a value is an instance of a class or its subclass.	class-system.fs	development/scale/tests.fs
`define`	Defines a user word globally by splicing its body.	control.fs	development/scale/tests.fs
`duplicate`	Duplicates the top stack value.	stack.fs	development/scale/tests.fs
`swap`	Swaps the top two stack values.	stack.fs	development/scale/tests.fs
`drop`	Drops the top stack value.	stack.fs	development/scale/tests.fs
`over`	Copies the second stack value to the top.	stack.fs	development/scale/tests.fs
`rotate`	Rotates the third stack value to the top.	stack.fs	development/scale/tests.fs
`add`	Adds two numbers.	evaluator.fs	development/scale/tests.fs
`subtract`	Subtracts two numbers.	evaluator.fs	development/scale/tests.fs
`multiply`	Multiplies two numbers.	evaluator.fs	development/scale/tests.fs
`divide`	Divides two numbers; zero is a named refusal.	evaluator.fs	development/scale/tests.fs
`lessThan`	Compares two numbers.	evaluator.fs	development/scale/tests.fs
`greaterThan`	Compares two numbers.	evaluator.fs	development/scale/tests.fs

Word	Purpose	Definition	Used in
`equals`	Compares two values.	evaluator.fs	development/scale/tests.fs
`print`	Appends output to the actor's output.	evaluator.fs	development/scale/tests.fs

Expression IR
The **Expression IR** is `ExpressionAst`: the surface/elaborated expression language. It holds literals, words, calls, pipelines, explicit partial application, quotations, blocks, `let`, `lambda`, `if`, `pattern-match`, declarations, conditions, restarts, generics, and messages. It is code, not values, and it is host-independent.

It includes a `PatternMatch` node whose clauses contain Pattern IR trees. That is composition, not a second expression language: expressions produce values; patterns bind names.

What belongs here:

1. application forms: `Call`, `Pipeline`, `ReversePipeline`, `Partial`;
2. value-producing forms: literals, `Quotations`, `Blocks`, `Let`, `Lambda`, `If`;
3. declaration and runtime forms: match, conditions, restarts, generic functions, messages;
4. source offsets and portable-document tags needed by diagnostics and tooling.

What does not belong here:

1. stack effects, slots, closure instructions, or kernel execution;
2. pattern binding modes, exhaustiveness, or pattern decision trees;
3. kernel image tags and ABI encoding.

The Expression IR lowers to the kernel/Prelude boundary: pipelines and partials desugar to calls or closures, then the lowering path produces `LoweredInstructions`, and the term runtime uses `TermIr`. The pattern sublanguage is linked as Pattern IR.

Why it is separate from Pattern IR: an expression answers "what value is produced?"; a pattern answers "which variables are bound, and is the shape possible?". Sharing one union would make an `if` representable in pattern position and a pattern representable as a value, forcing every consumer to partially validate. Keeping the two phases total is parse-don't-validate at the IR level.

Definition: expression-ast.fs, docs/general-ir.typ. Used in: prose-parser.fs, lowering.fs.

Pattern IR
The **Pattern IR** is `PatternAst`: the pattern sublanguage used by `ExpressionAst.PatternMatch` and by the pattern analyses. It holds wildcards, variables with usage modes (`Fresh`, `Repeated`, `Pin`, `Shadowing`), constructors, typed patterns, and literal patterns; guards and clauses live at the expression clause boundary.

What belongs here:

1. pattern shape and binding: `Wildcard`, `Variable`, `Constructor`, `TypedPattern`, literal patterns;
2. binding intent: fresh, repeated, pinned, shadowing;
3. pattern identity and source position for diagnostics;
4. the portable pattern document and its kinds/predicates.

What does not belong here:

1. expression evaluation, calls, lambdas, or control flow;

2. variable values or stack effects;
3. a whole program or a top-level expression.

The Pattern IR is embedded in the Expression IR, not a peer top-level IR: `ExpressionClause.Pattern` is a `PatternAst.Pattern`. It is consumed by matching, exhaustiveness/refutation analysis, shadowing and duplicate checks, the decision-tree compiler, the SMT bridge, and the portable pattern document. It lowers to the ordinary expression/lowered control flow; the kernel sees no pattern node.

Why it is separate from the Expression IR: a pattern binds names and has its own validity rules; an expression produces values and has effects. Merging them would couple unrelated schema versions, refusal types, and analyses, and would put pattern semantics into every expression consumer. The split lets pattern compilation and matching be tested and optimized without changing expression evaluation.

Definition: pattern-ast.fs, pattern-portable.fs, pattern.fs. Used in: expression-ast.fs, docs/specification.typ (7).

Term IR
The kernel's core executable representation, `TermIr`: literals, words, quotations, branches, slots, closures, and failure. `TermProgram` is the canonical `schema:1`, `form:"term-program"` document. The active program is a `Term list`; a quotation is a code value. It is the input to pattern and expression lowering, not a surface language. See Lowered IR for the compilation target and receipt for the verified form. Definition: term-ir.fs, docs/term-ir.typ. Used in: program.fs, tests.fs.

Lowered IR
`LoweredInstructions` is the compilation target: literals, local slots, branches, labels, word calls, closures, and failure. `LoweredAbstraction` executes it; the AOT writer emits it. It is not the source language and does not carry surface syntax. It is produced from Expression IR and shares the runtime value model with Term IR. Definition: lowered-ir.fs, lowered-abstraction.fs. Used in: lowering.fs, tests.fs.

Code value
A quotation value is `[QuoteTag; documentId]`; `documentId` names the canonical term-program document for the body. It is code, not token text: `apply` decodes it and splices its terms. It is the runtime form of a quotation and the value a Term IR `Quote` pushes. Definition: code.fs, docs/term-ir.typ. Used in: control.fs, tests.fs.

Receipt
`VerifiedTermProgram` is the opaque capability issued by `TermVerification`. It carries the program and a SHA-256 digest of its canonical bytes; the portable form is `term-receipt` JSON. `scale_verify` issues it; execution enforcement is separate. See Term IR and Kernel ABI. Definition: term-verification.fs, docs/term-ir.typ. Used in: abi.fs, tests.fs.

Microkernel / Prelude split
The architecture separates a small trusted **kernel** (the abstract machine, the C ABI, the image loader, the structural verifier, and the execution safety rules) from the **Prelude** (the language: word semantics, types, effects, classes, dispatch, conditions, actors, macros, and lowering). The kernel is the machine; the Prelude is the language. The split is not only a way to support several host languages.

The benefits, beyond one shared language core:

1. **Small trusted computing base.** The code that can corrupt the machine, allocate outside the image, or authorize execution is small enough to audit and to keep stable. Language growth does not enlarge the TCB.
2. **Independent evolution.** A new word, type rule, effect, class, pattern, or pipeline desugaring lands in the Prelude image. The kernel and its ABI version do not move unless machine behavior changes.
3. **Fault isolation and attribution.** A language bug is a Prelude failure with a Prelude layer; a kernel/ABI failure is a separate class. A supervised actor can fail and restart without taking down the kernel or its siblings.
4. **Security and capabilities.** The kernel performs no host I/O and knows no host semantics. Host calls, transport, files, and secrets are explicit capabilities at the adapter edge; the AOT kernel does not execute dynamic host code.
5. **Portability and conformance.** One portable Prelude image and one narrow ABI serve every adapter. A new host is a marshalling adapter plus the same conformance suite, not a second implementation of the language.
6. **Performance and specialization.** Hot machine primitives stay native; the Prelude can be swapped, JIT-ed, or specialized per host without an ABI break. The alternative is N hosts times M features of duplicated semantics.
7. **Deployment and live operation.** The Prelude is swappable data. A live instance can change code through the actor protocol, and rollback is a history operation; the kernel process need not restart.
8. **Testability.** Kernel boundary tests run without the language; Prelude tests run without an FFI; the differential corpus compares interpreter and lowered paths; conformance compares every adapter. The layers fail independently and visibly.
9. **Organizational scaling.** Kernel, language, verification, and adapter work can proceed on separate contracts and release cadence. The ABI header and the invariants are the coordination points, not a shared mutable codebase.
10. **Formal reasoning at the right layer.** The kernel has a small operational semantics; the Prelude owns the rich type rules; SMT/domain verification is a third property. A proof or verifier does not have to model the whole host.
11. **Incremental migration.** The F# Prelude remains the reference while the Scale Prelude is ported feature by feature. Both can run against the same tests, and a feature can fall back during the transition.
12. **Heterogeneous actors and IPC.** Instances can run different Preludes or host languages while sharing one actor wire protocol. Language-level send/receive, remote handles, monitors, and supervision cross the boundary as data, not as a shared host heap.
13. **Observability and proof.** The verifier issues a receipt bound to the canonical term bytes; the image is append-only; errors are named values. The system can say which layer refused, which program was verified, and what changed.
14. **Language pluralism.** Pattern matching, pipelines, prose, and future front-ends are surface frameworks that elaborate into the same core terms. The core does not grow a case per surface syntax.
15. **Multi-tenancy and resource policy.** The kernel owns the machine and the adapter owns the effects, so isolation, quotas, deadlines, and supervision can be enforced without giving each language its own runtime.

Definition: docs/term-ir.typ (kernel/Prelude split), docs/INVARIANTS.typ. Used in: docs/specification.typ, docs/decision-records.typ (D95, D160).

Pattern-matching framework

Pattern matching is a feature-specific framework, not an ad hoc evaluator branch. It has its own `PatternAst`, a portable pattern document, a lowering path, and a decision procedure. The framework buys:

1. **One syntax, many consumers.** The parser, interpreter, lowering, verifier, emitter, and diagnostics share one pattern tree instead of each re-deriving clauses from text.
2. **Portable, versioned patterns.** A pattern document is host-independent data with a schema and kinds/predicates. It can cross an actor or instance boundary and be re-checked on arrival.
3. **Analysis before execution.** Exhaustiveness, refutation, coverage, clause order, duplicate variables, shadowing, pins, repeated variables, and guards are checked as named failures. A missing case is not a silent false.
4. **Compilation and parity.** A pattern can lower to a decision tree or lowered instructions, while the interpreter remains the reference. The differential corpus compares the two.
5. **Extensibility as data.** Constructors, kinds, predicates, and guards are image entries. A new pattern form does not require a new kernel case.
6. **Verification bridge.** Patterns carry enough structure for SMT obligations, witnesses, and undecided outcomes; the domain verifier stays separate from matching.
7. **Error attribution.** Pattern failures carry the clause, pattern, offset, and rule, so a refusal names the next action.
8. **Testing leverage.** Pattern vectors, malformed documents, property round-trips, and differential cases test the framework without running a whole program.

Definition: pattern-ast.fs, pattern-portable.fs, pattern.fs. Used in: docs/specification.typ (7), tests.fs.

Pipeline framework

A pipeline is a feature-specific surface framework with an explicit `Pipeline` node and style, elaborated to calls. It is not the concatenative stack and it is not Elixir's macro semantics. The reverse pipe `<|` is supported for F# package compatibility and is discouraged; `f <| x` is `f(x)` and the value is on the right. The framework buys:

1. **1:1 F# semantic fidelity.** The piped value is inserted as the **last** argument, the operator is left-associative, and it binds looser than a call. Imported F# code keeps its meaning.
2. **Style as data.** `LastArgument` and `FirstArgument` are explicit and language-independent. An F#-compatible surface uses the default last position; an Elixir-compatible surface uses the first position. Neither requires a kernel change or forks the core.
3. **Surface preservation.** The `Pipeline` node keeps the source offset and style in the portable expression document; lowering desugars it to `Call`. Diagnostics can name the stage, not just the final call.
4. **Reuse across front-ends.** Prose, a future F#-syntax surface, and a host DSL can share one pipeline semantics and one portable document.
5. **Static analysis and optimization.** An explicit pipeline exposes stage arity, argument order, effects, and dataflow. A compiler can type-check, fuse, or schedule stages; the value is evaluated once.
6. **Testability against references.** Golden ASTs, desugaring equality, evaluation order, argument position, and differential runs against F# make semantic drift visible.
7. **Error attribution.** A missing stage, a non-callable stage, and an arity mismatch are distinct refusals at the stage boundary.
8. **Evolution without kernel changes.** `|>`, operator sections, Elixir
9. style, or a pipeline macro are Prelude/front-end work; the kernel term set
10. stays the same.

11. **Effect and capability policy.** Pipeline stages are ordinary calls, so the existing effect/capability checks apply uniformly; a pipeline cannot hide an effect in a special evaluator path.

The current boundary is explicit: a pipeline stage may carry whitespace arguments (`x |> f a`), and a call may use explicit holes (`subtract(_ 10)`). General automatic currying is not expressible; the holes are the partial-application syntax. That is a declared limitation, not a silent rewrite.

Definition: expression-ast.fs, prose-parser.fs, lowering.fs. Used in: docs/specification.typ (3.10), docs/prose-surface.typ.

Kernel ABI
The single C ABI contract in `scale_abi.h`: nine exports, one integer version (currently 1), one independent handle per instance, callback table, status codes, and image transfer. It is the boundary between the kernel and a host adapter version equality is the compatibility rule. See receipt for `scale_verify`. Definition: scale_abi.h, docs/kernel/abi.typ. Used in: scale_adapter.c, tests.fs.

Host adapter
The marshalling edge for one host language: loads the kernel, checks the ABI, allocates callback buffers, and implements transport. It contains no Scale semantics. The C adapter is the reference; the Elixir harness is a wire peer, not a language adapter. See implementation instance. Definition: components/adapter-c/README.md, docs/adapters/README.typ. Used in: conformance.c, adapters/elixir.

Implementation instance
A running Scale instance with an actor scheduler, connected to other instances by the actor wire. S1 is the numeric envelope, S2 the worker capability and supervision; language-level send uses a remote handle. See Actor. Definition: actor-host.fs, docs/actor-ipc.typ. Used in: language-transport.fs, granular-tests.fs.

Actor wire
The frozen Cap'n Proto-compatible wire subset: one segment, a fixed data/pointer layout, and send/reply/monitor operations. Reference vectors and the Elixir harness pin the bytes; far pointers and multi-segment messages are out. See implementation instance and remote handle. Definition: wire.fs, docs/binary-format.typ. Used in: components/ipc/vectors, adapters/elixir.

Remote handle
A registered capability `[RemoteHandleTag; instance; capability]` naming an actor in another implementation instance. The language records a transport request; the host resolves and sends it. A tagged tensor the host did not register is not a capability. Definition: scheduler.fs, docs/actor-ipc.typ. Used in: language-transport.fs, granular-tests.fs.

Adjustable semantic construct
A semantic fact at a boundary that Scale can classify, verify, and—only with a declared reason—override. The point is not to make semantics arbitrary. It is to keep a foreign definition's real semantics auditable while leaving an ordinary package call simple.

Why the split. A package call is black-box: the callee owns recursion, initialization, diagnostics, and its body. The caller needs a fixed **call contract**: exported name, arity, parameter and result types, effects and errors, marshalling, and package version/ABI. It must not infer F#'s `let rec` from the call edge, and Elixir calls have no equivalent definition-site classification. Importing an F# definition is white-box: Scale is taking responsibility for the definition-site semantics, so it must record whether the binding is non-recursive, self-recursive, or mutually recursive, how it initializes,

and which diagnostics apply. Only the import/port lane needs the adjustable machinery; the call lane stays a fixed contract.

How it works. The default is automatic: read FCS's classification (`let`, `let rec`, `let rec ... and ...`) and lower it to the Scale definition marker. The opt-in explicit tier adds a version-controlled package manifest keyed by binding; it still runs FCS and compares. Agreement is silent. Disagreement is a diagnostic. Disagreement with a declared reason is accepted and tracked; disagreement without a reason is a build error. Package modes are pure automatic, mixed, and strict explicit. Both tiers emit one `BindingClassification` record with name, kind, calls in body, source range, and declared reason; the record has no tier field.

The constructs.

1. **Tier A automatic classification** — FCS read-off is the default; no manifest. See function-values.typ.
2. **Tier B explicit override** — per-binding manifest declaration, checked against FCS; a disagreement requires a declared reason. See D172.
3. **Package modes** — pure automatic, mixed, strict explicit; a per-package annotation, not a language fork.
4. **BindingClassification** — the versioned record shared by the definition words and the IPC bridge; it has no tier field.
5. **Definition markers** — `logic` is non-recursive and refuses a self-call; `cycle` is recursive and uses the plain name; `and` groups cycle definitions. This is a build-time check, not a convention. See function-values.typ.
6. **Initialization phase** — image construction checks definition and value initialization order before execution begins. See function-values.typ.
7. **Diagnostic mapping** — FS0040, the unused-`rec` warning, and the value restriction are represented with Scale source ranges. See D172.
8. **Import/port lane** — the only lane that consumes Tier A/B, classification, initialization, and diagnostics. F# source port: prelude-port.typ feature gate: language-feature-ir.typ.
9. **Scale definition markers** — Scale's own `logic`/`cycle` recursion contract, separate from package classification; registered in D172.
10. **Fixed call contract** — exported name, arity, types, effects, marshalling, and version/ABI. This is intentionally **not** adjustable. See kernel ABI, host adapter, implementation instance, actor wire, remote handle, and conformance.

The IPC bridge may carry classification metadata `(name, kind, version)`, but invocation still uses the fixed call contract. The bridge never serializes a function body. Elixir calls use the fixed contract only; F# classification is meaningless at an Elixir call edge.

Definition: docs/function-values.typ, docs/decision-records.typ (D172), docs/prelude-port.typ. Used in: docs/prelude-port.typ, docs/language-feature-ir.typ, this glossary.

Prose parser
The controlled-prose expression parser: match/clauses, calls, pipelines, explicit partial holes, and source positions. It produces Expression IR and is distinct from the Basic English Parser. Definition: prose-parser.fs, docs/prose-surface.typ. Used in: tests.fs, bases/scale-cli/entry.fs.

Portable document
A canonical JSON envelope with a schema version and form, carrying no host objects. The forms are pattern, expression, package, image, term-program, and term-receipt. The term and pattern documents are the portable forms of Term IR and Pattern IR the expression document is the portable

form of Expression IR. Definition: portable-json.fs, term-portable.fs. Used in: expression-portable.fs, pattern-portable.fs.

Conformance
The C harness that loads a kernel through an adapter, checks the ABI version, initialises an image, exercises a declared primitive, checks the result, and tears down. It compiles in alpha and runs at release. See host adapter. Definition: conformance.c, docs/test-strategy.typ. Used in: components/adapter-c/README.md.

Decision record
A historical, numbered note in `docs/decision-records.typ` explaining one semantic choice. The heading is `D<n>`; the body records `Decision`, `Why`, `Boundary`, `Status`, and `Non-goals` (older records may use `Options`, `Recommendation`, and `Unblocks`). Records are append-only: a later record supersedes an earlier one, numbers are never reused, and old status/count snapshots are not rewritten. Live status is the commitments ledger. Definition: docs/decision-records.typ. Used in: docs/commitments-ledger.typ, docs/specification.typ.

Image document
The canonical, versioned Prelude image JSON: name, version, source hash, kinds, predicates, units, exports, imports, capabilities, and primitives. It is data, not the Store execution store, and its schema version is checked at load. See Kernel ABI. Definition: prelude-image.fs, docs/binary-format.typ. Used in: abi.fs, tests.fs.

Historical aliases
Older prose may still use these names. The current name is on the right; each current name links to its entry above.

Old name	Current name
`Image` / `Heap`	Image document / Store
`ClassTag` / `class-of` / `TypeVerdict`	ValueTag / value-of / ValueVerdict
`model` / `admit`	set / add-member
`Vivid`	SetAlgebra
`Lace`	ScalarRead
`Bag`	VocabularyList
`Probe`	QuotationRun
`Term` (old)	UnificationTerm current `Term` is Term IR
`Closure`	ProvenClosure
`Event` / `Step`	ChannelEvent / HistoryStep
`Guard`	RefinementGuard / RuntimeGuard
`FsharpName`	ImplementationName
`Spec`	Specification
`withNack` / `nack`	with-fallback / fallback
`Reifier` / `reification`	Refiner / refinement
`snoc`	`appendElement`
`BAG`	Vocabulary
`FixedSpec`	`FixedSpecification`

Old name	Current name
`zonk`	“apply the substitution” in prose
`Lia` as the prototype	Lia is the mascot; Scale is the product

Tools

The external tools the index names. Each is linked to its own documentation.

Tool	Purpose	Documentation
SageFs	The warm F# REPL and MCP inner loop.	docs/sagefs.typ
MCP	The protocol that exposes SageFs and .NET tools.	modelcontextprotocol.io
LSP	Language-server navigation and diagnostics.	language-server-protocol
Typst	The documentation typesetting system.	typst.app
Tinymist	The Typst language server.	tinymist
Hedgehog	Property generation and shrinking.	fsharp-hedgehog
Z3	The SMT solver behind the obligations.	Z3Prover/z3
Tokenfold	Lossless-first shell and payload compression. Default is Tokenfold-only; RTK is optional only when raw capture is available. A wrapper failure is a tool failure, not a license to rerun raw.	AGENTS.md §5
Fantomas	The F# formatting authority.	fsprojects/fantomas
just	The recipe runner for the gate and the count update.	casey/just